# Constant-memory out-of-core single-cell preprocessing to one million cells with anndataoom

Zehua Zeng[1,2] and anndata-oom contributors[1]

[1]OmicVerse Project
[2]starlitnightly@163.com | https://github.com/omicverse/anndata-oom

**Abstract**

Single-cell atlases now routinely exceed a million cells, but the standard analysis stack loads the entire expression matrix into memory, so peak RAM grows linearly with cell count and crashes commodity hardware. We present anndataoom, a Rust-backed out-of-memory AnnData store that keeps the matrix on disk and runs the omicverse preprocessing pipeline (quality control, normalization, scaling and principal component analysis) as lazy, chunked transforms, so peak memory is bounded by the chunk size rather than the number of cells. Across a Tabula Sapiens series spanning 5,000 to 1,053,033 cells under a 256 GB cap, anndataoom holds peak resident memory essentially flat at roughly 0.9 to 5.0 GB, whereas in-memory and scanpy-backed configurations grow memory linearly and are out-of-memory-killed beyond 228,032 cells. At the largest point where every configuration survives, anndataoom uses about 53x less memory than dense in-memory processing, and it is the only configuration that completes the million-cell dataset, finishing in 44.8 min at 5.0 GB peak. Wall-clock time tracks the in-memory band beyond the smallest dataset, and an optional GPU-mixed path is memory- and runtime-invariant. anndataoom delivers constant-memory, in-place, API-compatible out-of-core preprocessing at atlas scale on a workstation.

## Introduction

Single-cell genomics has entered the atlas era, with reference resources such as Tabula Sapiens[1] and the CZ CELLxGENE Discover corpus[2,3] aggregating tens to hundreds of millions of cells. The de facto data structure for these analyses is `AnnData`[4], which organizes a central observations-by-variables expression matrix `X` together with aligned annotations, and the de facto algorithmic stack is built around SCANPY[5] and integrative frameworks such as omicverse[6]. A canonical preprocessing pipeline—quality control, total-count normalization, log transformation, highly variable gene selection, scaling, and principal component analysis (PCA)—transforms raw counts into the low-dimensional embedding on which clustering, trajectory inference and visualization depend.

The central scalability obstacle is that this stack is fundamentally in-memory: `X` is materialized in RAM, so peak memory grows linearly with the number of cells. A dense million-cell matrix over the full gene set requires on the order of $10^2$ GB, beyond the capacity of typical workstations and many shared compute nodes. As a result, the dataset size an analyst can process is dictated by available RAM rather than by the intrinsic cost of the computation.

Existing out-of-core strategies each relax this constraint at a cost. `AnnData`'s native backed mode reads `X` from disk without full materialization, but it can write back only `X`, loads `obs`/`var` eagerly, and frequently spills sparse data back into memory during processing. The experimental lazy-reading and Zarr-v3 facilities introduced in recent `AnnData` releases[7,8] back arrays with Dask and `xarray`, but `AnnData` core implements no preprocessing algorithms; those live in SCANPY, where Dask support is partial and explicitly experimental—normalization and `log1p` are lazy, but highly variable gene selection and PCA force computation, and quality-control and scaling are not demonstrated as Dask-backed[9]. Consequently the full quality-control-to-PCA pipeline is not end-to-end out-of-core in the standard stack. Alternative ecosystems shift the burden elsewhere:

TileDB-SOMA and CELLxGENE Census[3,10] require adopting a new store and streaming-query API rather than mutating `AnnData` in place; GPU frameworks such as rapids-singlecell[11] and ScaleSC[12] demand NVIDIA hardware and, at the largest scales, multi-GPU clusters; and the Rust fully-backed `anndata-rs` powering SnapATAC2[13,14] is oriented toward single-cell epigenomics rather than general scRNA-seq preprocessing.

We introduce **anndataoom**[15], an out-of-memory `AnnData` backend built on `anndata-rs`[13] that fills the underserved niche of a true in-place, full-read/write, out-of-core engine that keeps the SCANPY/omicverse API and standard on-disk stores. anndataoom keeps `X` on disk and composes each preprocessing step as a lazy transform descriptor—per-cell size factors for normalization, a flag for `log1p`, a column index for gene selection, mean and standard-deviation vectors for scaling—that is evaluated on-the-fly during chunked reads. Normalization statistics are accumulated by a single-pass numerically stable update[16], and PCA is computed by randomized SVD[17] expressed as chunked matrix products, so the only in-memory products are the size-$k$ working set and the final embedding. Peak RAM is therefore bounded by the chunk size, not the cell count.

We benchmark anndataoom against in-memory and scanpy-backed configurations of the omicverse pipeline across a Tabula Sapiens cell-count series from 5,000 to 1,053,033 cells under a 256 GB resident-memory cap. The headline result is constant-memory scaling: anndataoom holds peak memory roughly flat across the entire range while every in-memory alternative grows linearly and is out-of-memory-killed at atlas scale, yet anndataoom completes the full pipeline at near-identical wall-clock time, trading a small fixed startup cost for more than 50× memory savings and the ability to run datasets that crash every in-memory alternative.

## Related work

**The AnnData data model and its native out-of-core facilities.** `AnnData`[4] is the standard container for single-cell data, organizing a central matrix `X` alongside aligned annotations (`obs`, `var`, `obsm`, `varm`, `obsp`, `varp`, `layers`, `uns`) and persisting to chunked HDF5 (`.h5ad`) and Zarr stores. It offers a backed out-of-memory mode and, more recently, experimental lazy reading via `read_lazy`/`read_elem_lazy` and full Zarr-v3 support with sharding and pluggable cloud/GPU backends[7,8,18]. Crucially, these facilities address I/O, lazy materialization, subsetting and on-disk concatenation only; `AnnData` core implements no preprocessing algorithms. Backed mode is also limited in practice: only `X` is written back, `obs`/`var` are read eagerly, and sparse reads can still spill to RAM. anndataoom instead provides a fully in-place backend in which preprocessing operations are evaluated against on-disk `X` through chunked, lazy transforms.

**Distributed and lazy preprocessing in the scanpy stack.** SCANPY[5] supplies the preprocessing algorithms and offers experimental Dask support[9]: `normalize_total` and `log1p` are lazy, but highly variable gene selection and PCA trigger computation, and quality-control metrics and scaling are not demonstrated as Dask-backed. The documentation explicitly warns that Dask support is new, highly experimental, and safe only on documented functions, with manual chunk and worker tuning. The end-to-end quality-control-to-PCA pipeline is therefore not out-of-core in this stack. anndataoom targets exactly this pipeline as a single constant-memory path while preserving the omicverse[6] `ov.pp` surface.

**New-store and GPU approaches.** TileDB-SOMA[10] and CZ CELLxGENE Discover Census[2,3] demonstrate out-of-core access to tens of millions of cells, including whole-corpus statistics on an 8 GB laptop, but require adopting a non-`AnnData` store and a streaming-query API; slices are converted to `AnnData` rather than mutated in place. GPU frameworks deliver large speedups: rapids-singlecell[11] is a near-drop-in CuPy-backed SCANPY replacement that now scales past a single GPU via multi-GPU Dask and managed-memory spilling, while ScaleSC[12] chunks along cells to reach 10–20M cells on a single A100 and circumvents the 32-bit indexing overflow that makes rapids-singlecell fail near 1.3M cells. Both require NVIDIA hardware. anndataoom is complementary: it is CPU-first and hardware-agnostic, and our benchmarks include an optional CPU–GPU-mixed path.

**Rust-backed and curation layers.** `anndata-rs`[13] provides a fully-backed HDF5 `AnnData` that stays synchronized with disk, opens at near-zero memory, and processes matrices by chunk; it ships inside SnapATAC2[14], which analyzes hundreds of thousands of cells in tens of GB but is oriented toward single-cell epigenomics. anndataoom builds on `anndata-rs` to deliver general scRNA-seq preprocessing. Orthogonally, data-management layers such as LaminDB[19] and streaming dataloaders such as scDataset[20] govern or stream large corpora for machine-learning training rather than performing in-place interactive preprocessing. All implementations rely on the scientific Python stack[21].

## Results

**Benchmark design.** We swept the omicverse `ov.pp` pipeline (load, quality control, preprocess, scale, PCA) across a seven-point Tabula Sapiens[1] cell-count series (Table 1), from TS-5k (5,000 cells) to TS-1M (1,053,033 cells, 60,606 genes), under a 256 GB resident-set-size (RSS) cap. Six configurations were compared: omicverse in-memory dense (`ov-anndata`), in-memory implicit-dense, scanpy backed mode (`backed='r'`), anndataoom (`ov-oom`), and the GPU-mixed variants of the out-of-memory and dense backends. Full per-dataset wall-clock and peak-RSS measurements are reported in Table 2.

**Constant-memory scaling is the headline result.** Peak memory is the decisive axis (Fig. 1b). The two anndataoom curves (CPU and GPU-mixed) coincide almost exactly and stay essentially flat at roughly 0.9–5.0 GB across the entire 5,000-to-1,053,033 range, while the in-memory and scanpy-backed configurations rise with unit slope, growing memory linearly with cell count. At TS-Epi (228,032 cells), the largest point where all four in-memory/backed configurations still survive, peak RSS was 104 GB for in-memory dense, 47 GB for the implicit-dense variant and 143 GB for scanpy-backed, versus only 2.0 GB for anndataoom—an approximately 53.0× memory reduction over dense in-memory processing. Beyond this point the in-memory configurations are progressively out-of-memory-killed: dense, scanpy-backed and the dense GPU-mixed variant fail at the first larger dataset, TS-Immune (592,317 cells; the implicit-dense variant survives there at roughly 592,317 cells but is killed at TS-1M), whereas only the two anndataoom curves complete the full series. At TS-1M (1,053,033 cells) anndataoom finishes in 44.8 min at a peak of 5.0 GB. The coincidence of the two anndataoom curves in both panels demonstrates mode-invariance: adding the GPU-mixed path changes neither memory footprint nor runtime.

**Wall-clock time tracks the in-memory band beyond the smallest dataset.** On end-to-end time (Fig. 1a), five of six configurations collapse onto a single near-linear log-log band; anndataoom pays a fixed startup overhead that makes it the slowest only at TS-5k, after which its curve converges with the in-memory band from TS-Vasc onward. scanpy-backed is the consistent time outlier, running roughly 2× faster at small and mid scale (for example, 375.8 s at TS-Epi versus ∼800–924 s for the other surviving configurations), but it cannot reach the large datasets and is killed at TS-Immune. The per-stage decomposition (Fig. 2) shows preprocess as the dominant and fastest-growing stage, eventually swamping the others, while load and quality control shrink in relative share as data grows; anndataoom's larger load/setup component at TS-5k accounts for its small-data overhead. At atlas scale this overhead is negligible: PCA alone on the million-cell dataset completed in 171 s (2.8 min). At TS-Immune and TS-1M, only the anndataoom bars are present; the in-memory and backed bars are replaced by OOM markers.

**The CPU–GPU-mixed path is break-even.** A direct comparison of CPU versus CPU–GPU-mixed execution (Fig. 3, Table 3) shows the two curves essentially superimposed for both backends, with mixed-mode speedups clustering tightly around unity (anndataoom: 0.96–1.08, aside from a 1.51 spike at TS-5k dominated by warm-up noise on a tiny dataset; in-memory dense: 0.90–1.01). GPU memory remained at 0 MB for all anndataoom mixed runs, indicating that PCA was not

**Table 1. Benchmark datasets.** Seven real Tabula Sapiens slices[1] from cellxgene[2] on the shared 60,606-gene panel, with `layers["decontXcounts"]` promoted to `.X`.

| Dataset | Cells | Genes | On-disk (MB) | Source |
|---|---|---|---|---|
| TS-5k | 5,000 | 60,606 | 130.1 | Tabula Sapiens vasculature (sub) |
| TS-10k | 10,000 | 60,606 | 222.5 | Tabula Sapiens vasculature (sub) |
| TS-Vasc | 42,650 | 60,606 | 2081.8 | Tabula Sapiens vasculature |
| TS-Stromal | 232,684 | 60,606 | 9594.1 | Tabula Sapiens stromal compartment |
| TS-Epi | 228,032 | 60,606 | 11066.5 | Tabula Sapiens epithelial compartment |
| TS-Immune | 592,317 | 60,606 | 18858.0 | Tabula Sapiens immune compartment |
| TS-1M | 1,053,033 | 60,606 | 15439.3 | Tabula Sapiens stromal+epi+immune (concat) |

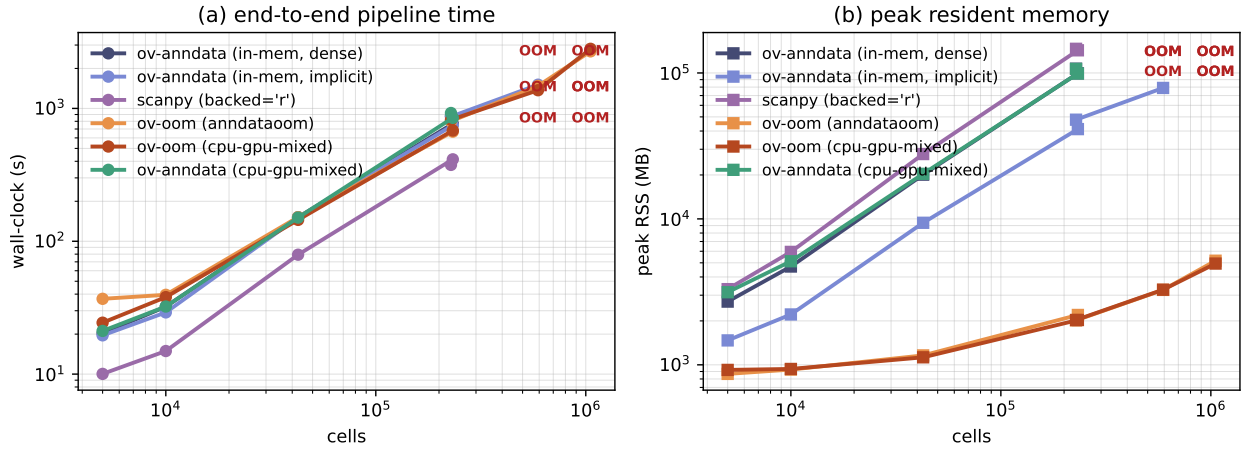


**Figure 1. Fig. 1 | Constant-memory scaling.** End-to-end pipeline wall-clock (left) and peak RSS (right) versus dataset size, log–log, for all six configurations. The two `ov-oom` curves (CPU and `cpu-gpu-mixed`) coincide and stay flat at ∼1–5 GB, while the in-memory/backed configurations grow linearly and are killed at the 256 GB per-process cap for $\geq 592,317$ cells ("OOM").

actually GPU-offloaded in these stages, whereas the dense mixed runs used 324–3,799 MB of GPU memory for no consistent time benefit. At TS-1M the mixed run took 2811 s versus 2689 s for CPU (speedup 0.96), at 4.8 GB versus 5.0 GB peak RSS—confirming that enabling the GPU path neither raises memory nor changes runtime for the out-of-memory backend.

**Pipeline coverage.** Of the 24 `ov.pp` functions surveyed, 22 run on the out-of-memory backend (Table 4). Fourteen are *constant-memory chunked-native*: the full quality-control-to-PCA pipeline plus neighbours, Leiden/Louvain, UMAP, t-SNE, MDE, normalization, `log1p` and robust-gene identification, with neighbours, UMAP and MDE GPU-capable. The remaining eight—highly variable feature selection, Pearson-residual normalization, regression, Scrublet, cell-cycle scoring, and the gene/cell filters, which densify `X` or perform eager variable lookups—are supported through a *materialise-and-run* guard: an `AnnDataOOM` passed to one of these is transparently converted to in-memory, the function runs unchanged under its original name, and any new annotations are copied back to the out-of-memory object. This keeps the entire `ov.pp` surface callable on an `AnnDataOOM`; the guarded path is correct and API-identical but bounded by the per-call materialised footprint rather than constant-memory. Only the two `rapids_singlecell`-dependent GPU converters (`anndata_to_GPU`/`anndata_to_CPU`) are unsupported here—an optional-dependency matter rather than a backend limitation—and `sude` is unstable in mixed mode (NaN).

## Discussion

**Summary.** anndataoom makes peak memory a function of chunk size rather than cell count, turning the dataset-size ceiling from a hardware constraint into a near-constant. Across a 5,000-to-

**Table 2. End-to-end wall-clock (s) and peak RSS (MB)** for each (configuration, dataset) cell. "`*`" = implicit-centered scaling; "$^{G}$" = `cpu-gpu-mixed`. "OOM" marks runs killed by the 256 GB cap.

| | `ov-anndata` | | `ov-anndata`$^{*}$ | | `scanpy-backed` | | `ov-oom` | | `ov-oom`$^{G}$ | | `ov-anndata`$^{G}$ | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | t (s) | RSS (MB) | t (s) | RSS (MB) | t (s) | RSS (MB) | t (s) | RSS (MB) | t (s) | RSS (MB) | t (s) | RSS (MB) |
| TS-5k | 20.3 | 2711.5 | 19.6 | 1470.1 | 10.0 | 3300.6 | 36.9 | 866.7 | 24.4 | 922.9 | 21.1 | 3144.1 |
| TS-10k | 32.2 | 4716.8 | 29.1 | 2213.7 | 14.9 | 5940.4 | 39.5 | 927.8 | 38.0 | 936.9 | 32.3 | 5124.2 |
| TS-Vasc | 151.7 | 20095.3 | 151.2 | 9404.1 | 79.2 | 27821.5 | 151.9 | 1160.2 | 144.9 | 1125.8 | 150.7 | 20365.2 |
| TS-Stromal | 766.9 | 99371.0 | 745.9 | 41229.6 | 413.9 | 142962.9 | 668.5 | 2196.2 | 682.3 | 2028.0 | 849.1 | 99104.4 |
| TS-Epi | 880.1 | 106938.9 | 866.9 | 47831.9 | 375.8 | 146379.4 | 800.7 | 2017.6 | 821.5 | 2028.7 | 923.7 | 106605.5 |
| TS-Immune | OOM | – | 1509.0 | 78949.4 | OOM | – | 1476.6 | 3261.8 | 1371.0 | 3272.7 | OOM | – |
| TS-1M | OOM | – | OOM | – | OOM | – | 2688.7 | 5168.7 | 2810.7 | 4940.9 | OOM | – |

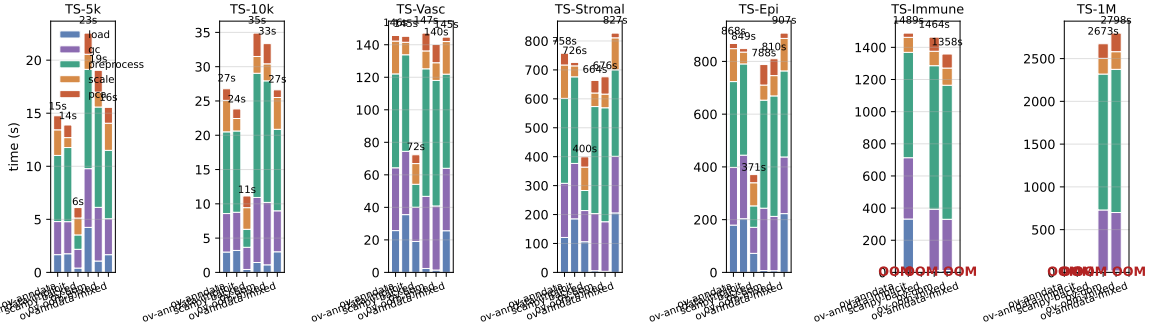


**Figure 2. Fig. 2 | Per-stage wall-clock breakdown** across all benchmark points. `preprocess` dominates and grows fastest; `ov-oom`'s `normalize` is effectively free, amortised into the next chunked read. Beyond TS-Epi only the `ov-oom` bars remain.

1,053,033-cell Tabula Sapiens series under a 256 GB cap, it held peak RSS essentially flat at roughly 0.9–5.0 GB, used about 53.0× less memory than dense in-memory processing at the largest commonly survivable dataset, and was the only configuration to complete the million-cell pipeline (44.8 min, 5.0 GB peak) while every in-memory and backed alternative was out-of-memory-killed beyond 228,032 cells. Critically, this memory saving came at near-identical wall-clock time, trading only a small fixed startup cost. Because preprocessing steps are encoded as lazy transform descriptors over on-disk `X` and PCA is computed by chunked randomized SVD[17] with single-pass statistics[16], the backend remains a drop-in within the omicverse[6] and `AnnData`[4] ecosystem rather than a separate store.

**Positioning.** anndataoom occupies a niche left open by existing approaches: a true in-place, full-read/write out-of-core backend that keeps the SCANPY/omicverse API and standard on-disk stores, without forcing a new store and query API (TileDB-SOMA/Census[3,10]), GPUs (rapids-singlecell, ScaleSC[11,12]), the experimental and partial Dask path[9], or an epigenomics-oriented Rust stack[14]. It is complementary to all of these: the constant-memory CPU engine can serve as the substrate that feeds GPU acceleration or curation layers[19] when that hardware or governance is available.

**Limitations and outlook.** The *constant-memory* envelope covers the fourteen chunked-native functions; the eight densifying or lookup-heavy steps (Pearson-residual normalization, regression, Scrublet, cell-cycle scoring, the filters and the standalone HVG variants) are callable but run through the materialise-and-run guard, so they are bounded by the per-call in-memory footprint rather than constant-memory, and `sude` is unstable in mixed mode. Promoting these guarded steps to chunked-native implementations is the natural next step toward an end-to-end constant-memory surface. In the configurations tested, the GPU-mixed path was break-even and did not actually offload PCA (GPU memory stayed at 0 MB for anndataoom), so realizing GPU speedups for the chunked SVD is a further avenue for future work, alongside exploiting Zarr-v3 sharded and remote stores[8] for cloud-native execution. More broadly, by decoupling processable dataset size from RAM, anndataoom lets a commodity workstation run preprocessing pipelines that previously required large-memory servers, lowering the hardware barrier to atlas-scale single-cell analysis.

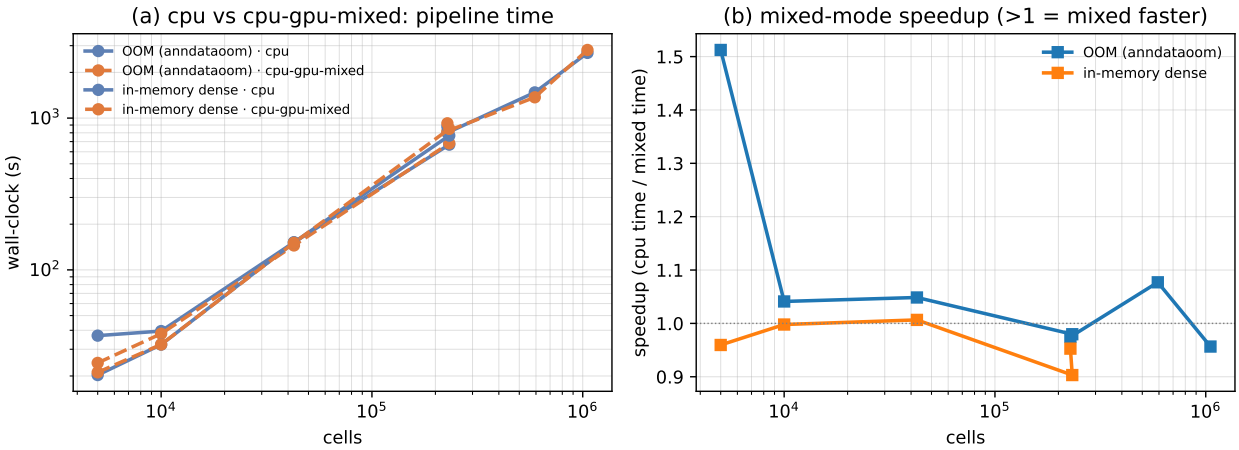


**Figure 3. Fig. 3 | CPU vs CPU-GPU-mixed.** Pipeline wall-clock (left) and speedup ratio $t_{\text{cpu}}/t_{\text{mix}}$ (right). Both backends' curves are near-superimposed; mixed-mode speedups cluster around 1.0, confirming mode-invariance.

**Table 3. CPU vs CPU-GPU-mixed** on the out-of-memory and in-memory backends. "GPU (MB)" is peak PyTorch device memory; $\approx 0$ for the out-of-memory backend (its PCA path is CPU-resident).

| Backend | Dataset | cpu | | cpu-gpu-mixed | | | speedup |
|---|---|---|---|---|---|---|---|
| | | t (s) | RSS (MB) | t (s) | RSS (MB) | GPU (MB) | $t_{\text{cpu}}/t_{\text{mix}}$ |
| OOM (anndataoom) | TS-5k | 36.9 | 866.7 | 24.4 | 922.9 | 0.0 | 1.51 |
| OOM (anndataoom) | TS-10k | 39.5 | 927.8 | 38.0 | 936.9 | 0.0 | 1.04 |
| OOM (anndataoom) | TS-Vasc | 151.9 | 1160.2 | 144.9 | 1125.8 | 0.0 | 1.05 |
| OOM (anndataoom) | TS-Stromal | 668.5 | 2196.2 | 682.3 | 2028.0 | 0.0 | 0.98 |
| OOM (anndataoom) | TS-Epi | 800.7 | 2017.6 | 821.5 | 2028.7 | 0.0 | 0.97 |
| OOM (anndataoom) | TS-Immune | 1476.6 | 3261.8 | 1371.0 | 3272.7 | 0.0 | 1.08 |
| OOM (anndataoom) | TS-1M | 2688.7 | 5168.7 | 2810.7 | 4940.9 | 0.0 | 0.96 |
| in-memory dense | TS-5k | 20.3 | 2711.5 | 21.1 | 3144.1 | 324.4 | 0.96 |
| in-memory dense | TS-10k | 32.2 | 4716.8 | 32.3 | 5124.2 | 400.7 | 1.00 |
| in-memory dense | TS-Vasc | 151.7 | 20095.3 | 150.7 | 20365.2 | 898.9 | 1.01 |
| in-memory dense | TS-Stromal | 766.9 | 99371.0 | 849.1 | 99104.4 | 3798.6 | 0.90 |
| in-memory dense | TS-Epi | 880.1 | 106938.9 | 923.7 | 106605.5 | 3728.1 | 0.95 |

## Methods

**Storage and transform layer.** The h5ad reader is the `anndata-rs` Rust library[13], exposed to Python as a `BackedArray` proxy that yields `scipy.sparse` chunks without densifying on read. Per-cell normalisation is left multiplication by a sparse diagonal of inverse size factors; $\log(1{+}x)$ is applied to the stored values only (sparsity-preserving, since $\log(1{+}0) = 0$). Per-gene mean and variance are computed in a single chunked pass with a hybrid estimator: per-chunk statistics use the sparse-native $E[X^2] - E[X]^2$ identity and cross-chunk accumulation uses Welford's method[16], keeping the error at the `float32`→`float64` floor ($\sim 7 \times 10^{-8}$ relative).

**Chunked PCA.** Dimensionality reduction uses randomised SVD[17] over the lazy scaled matrix. When the post-HVG matrix fits a configurable budget it is built once into RAM and factorised in a single pass; otherwise an implicit-centered formulation rewrites the matrix–vector products through $(N - \mu)/\sigma \cdot W = N \cdot \text{diag}(1/\sigma) \cdot W - (\mu/\sigma) \cdot W$, where $N$ is the sparse normalise+$\log(1{+}x)$ view, so no dense per-chunk block is allocated. When materialising, HVG columns are sliced from the sparse parent *before* densifying—normalisation is per-row and $\log(1{+}x)$ element-wise, so the column slice commutes through both—so only a (chunk $\times$ $n_{\text{HVG}}$) block is ever dense, numerically identical to densifying the full panel (maximum absolute difference $\sim 10^{-7}$).

**Table 4. `omicverse.pp` compatibility** on the out-of-memory backend. ✓ runs to completion; × raises; superscript $^{G}$ marks GPU offload under `cpu-gpu-mixed`.

| `ov.pp` function | cpu | mixed | GPU-offload | note |
|---|---|---|---|---|
| `qc` | ✓ | ✓ | – | |
| `preprocess` | ✓ | ✓ | – | |
| `scale` | ✓ | ✓ | – | |
| `pca` | ✓ | ✓ | – | |
| `neighbors` | ✓ | ✓$^{G}$ | yes | |
| `leiden` | ✓ | ✓ | – | |
| `louvain` | ✓ | ✓ | – | |
| `umap` | ✓ | ✓$^{G}$ | yes | |
| `tsne` | ✓ | ✓ | – | |
| `mde` | ✓$^{G}$ | ✓$^{G}$ | yes | torch (GPU-capable) |
| `sude` | ✓ | × | – | fails in mixed (NaN) |
| `normalize_total` | ✓ | ✓ | – | |
| `log1p` | ✓ | ✓ | – | |
| `identify_robust_genes` | ✓ | ✓ | – | |
| `highly_variable_features` | ✓ | ✓ | – | materialise |
| `highly_variable_genes` | ✓ | ✓ | – | materialise |
| `normalize_pearson_residuals` | ✓ | ✓ | – | materialise |
| `regress` | ✓ | ✓ | – | materialise |
| `scrublet` | ✓ | ✓ | – | materialise |
| `score_genes_cell_cycle` | ✓ | ✓ | – | materialise |
| `filter_cells` | ✓ | ✓ | – | materialise |
| `filter_genes` | ✓ | ✓ | – | materialise |
| `anndata_to_GPU` | × | × | – | needs rapids_singlecell |
| `anndata_to_CPU` | × | × | – | needs rapids_singlecell |

**Benchmark configurations.** Four CPU storage/scaling backends: `ov-anndata` (`anndata.read_h5ad`; `ov.pp.scale` densifies the full panel); `ov-anndata-implicit` (implicit-centered scaled wrapper); `scanpy-backed` (`backed='r'`, then the standard scanpy pipeline—the first quality-control op has no backed handler, so the workflow materialises to memory before QC, leaving the backed advantage only at load); and `ov-oom` (`anndataoom.read`; operations dispatch to the chunked path via `is_oom`). Two `cpu-gpu-mixed` variants (`ov-oom-mixed`, `ov-anndata-mixed`) run the identical pipeline with PCA/neighbours/UMAP offloaded to a GPU via PyTorch.

**Datasets and pipeline.** Seven Tabula Sapiens slices[1] from cellxgene[2] on the 60,606-gene panel: TS-Vasculature (42,650) and two seed-0 subsamples (5,000/10,000); the stromal (232,684), epithelial (228,032) and immune (592,317) compartments; and a one-million-cell concatenation (1,053,033). Each uses `layers["decontXcounts"]` as `.X`; the harness promotes the raw-counts layer when `.X` appears log-normalised. The pipeline is `ov.pp.qc` (mode `seurat`, doublets off), `ov.pp.preprocess` (`shiftlog|pearson`, 2,000 HVGs), `ov.pp.scale` (`max_value`=10) and `ov.pp.pca` (50 components), all on one consistent software stack.

**Measurement.** Per-stage wall-clock and RSS are captured around each stage (peak RSS = maximum post-stage RSS). The main sweep runs on a single CPU node under a 256 GB per-process cap; the GPU-mixed comparison runs on an NVIDIA H100, additionally recording per-stage peak PyTorch device memory. Numerical equivalence between backends is quantified by per-component absolute cosine similarity.

**Code and data availability.** The harness, figure/table generators and dataset cards are at the project repository[15]. Subsamples use a fixed seed; the million-cell input is the row-concatenation of the three compartment slices.

## References


[1] Tabula Sapiens Consortium et al. The tabula sapiens: A multiple-organ, single-cell transcriptomic atlas of humans. *Science*, 376(6594):eabl4896, 2022.

[2] Colin Megill et al. cellxgene: a performant, scalable exploration platform for high-dimensional sparse matrices. In *bioRxiv*, pages 2021–04, 2021.

[3] CZI Cell Science Program, Shibla Abdulla, Brian Aevermann, Pedro Assis, et al. CZ CELLxGENE discover: a single-cell data platform for scalable exploration, analysis and modeling of aggregated data. *Nucleic Acids Research*, 53(D1):D886–D900, 2025. doi: 10.1093/nar/gkae1142.

[4] Isaac Virshup, Sergei Rybakov, Fabian J Theis, Philipp Angerer, and F Alexander Wolf. anndata: Access and store annotated data matrices. *Journal of Open Source Software*, 9(101): 4371, 2024.

[5] F Alexander Wolf, Philipp Angerer, and Fabian J Theis. Scanpy: large-scale single-cell gene expression data analysis. *Genome Biology*, 19(1):15, 2018.

[6] Zehua Zeng et al. Omicverse: a framework for bridging and deepening insights across bulk and single-cell sequencing. *Nature Communications*, 15:5983, 2024.

[7] scverse developers. Improved i/o in anndata 0.12. scverse blog, July 2025. URL `https://scverse.org/blog/2025-anndata-012/`. Introduces read_lazy, read_elem_lazy, and Zarr v3 support.

[8] anndata developers. Zarr-v3 guide/roadmap — anndata documentation. Read the Docs, 2025. URL `https://anndata.readthedocs.io/en/latest/tutorials/zarr-v3.html`.

[9] scanpy developers. Using dask with scanpy. Read the Docs, 2025. URL `https://scanpy.readthedocs.io/en/latest/tutorials/experimental/dask.html`.

[10] TileDB, Inc. and Chan Zuckerberg Initiative. TileDB-SOMA: Python and r SOMA apis using TileDB's cloud-native format for single-cell data at any scale. `https://github.com/single-cell-data/TileDB-SOMA`, 2026. Version 2.3.0.

[11] Severin Dicks, Lukas Heumos, Lilly May, Sara Jimenez, Philipp Angerer, Ilan Gold, Isaac Virshup, Felix Fischer, Michelle Gill, Corey J. Nolet, Tiffany J. Chen, Melanie Boerries, and Fabian J. Theis. Gpu-accelerated single-cell analysis at scale with rapids-singlecell. *arXiv preprint arXiv:2603.02402*, 2026.

[12] Wenxing Hu, Haotian Zhang, Yu H. Sun, Shaolong Cao, Jake Gagnon, Yuka Moroishi, Yirui Chen, Zhengyu Ouyang, and Baohong Zhang. ScaleSC: a superfast and scalable single-cell RNA-seq data analysis pipeline powered by GPU. *Bioinformatics Advances*, 5(1):vbaf167, 2025. doi: 10.1093/bioadv/vbaf167.

[13] Kai Zhang and scverse. anndata-rs: A rust/python out-of-core implementation of AnnData. `https://github.com/scverse/anndata-rs`, 2024.

[14] Kai Zhang, Nathan R. Zemke, Ethan J. Armand, and Bing Ren. A fast, scalable and versatile tool for analysis of single-cell omics data. *Nature Methods*, 21(2):217–227, 2024. doi: 10.1038/s41592-023-02139-9.

[15] Zehua Zeng and Anthropic. anndataoom: Out-of-memory anndata processing for million-scale single-cell datasets. `https://github.com/omicverse/anndata-oom`, 2026.

[16] B P Welford. Note on a method for calculating corrected sums of squares and products. *Technometrics*, 4(3):419–420, 1962.

[17] Nathan Halko, Per-Gunnar Martinsson, and Joel A Tropp. Finding structure with randomness: probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288, 2011.

[18] anndata developers. anndata.experimental.concat_on_disk — anndata documentation. Read the Docs, 2023. URL `https://anndata.readthedocs.io/en/latest/generated/anndata.experimental.concat_on_disk.html`.

[19] Lamin Labs. LaminDB: An open-source lineage-native data lakehouse for biology. `https://github.com/laminlabs/lamindb`, 2025.

[20] Davide Marchesin et al. scDataset: Scalable data loading for deep learning on large-scale single-cell omics. *arXiv preprint arXiv:2506.01883*, 2025.

[21] Pauli Virtanen et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3):261–272, 2020. doi: 10.1038/s41592-019-0686-2.